

Dzień 2

Workloady, sieci, AuthN/AuthZ

Plan dnia

1. **Jobs / CronJobs**
2. **Storage:** Volumes, PV, PVC, StorageClass
3. **ConfigMaps + Secrets** (3 typy)
4. **AuthN/AuthZ:** Token, Cert, OIDC + RBAC
5. **Eksponowanie usług:** Gateway API + cert-manager
6. **StatefulSet, DaemonSet**

→ Repo: `day2/`

Jobs i CronJobs (D2/01-02)

Job = jednorazowe zadanie (DB migration, batch processing)

CronJob = harmonogram cron → tworzy Job

```
spec:
  parallelism: 5
  completions: 100
  backoffLimit: 6
```

Pułapki CronJob:

- `concurrencyPolicy: Forbid` — gdy poprzedni jeszcze trwa
- Joby kumulują się przy node down (`startingDeadlineSeconds`)

→ `day2/01_jobs/` , `day2/02_cronjob/`

Storage warstwy (D2/03)

	Co	Kiedy
<code>emptyDir</code>	temp per Pod	cache, scratch
<code>hostPath</code>	katalog z node	NIE produkcja
PV	klaster-level resource	admin pre-provisioning
PVC	user request	użycie z Pod
StorageClass	template dynamic provisioning	cloud (EBS, GCE disk, ...)

→ `day2/03_volume/`

ConfigMap (D2/05)

Dwa sposoby konsumpcji w Pod:

1. Env vars — **statyczne** (zmiana CM → restart Pod)
2. Mount jako pliki — **auto-refresh** ~60s

```
kubectl create configmap conf --from-file=./  
kubectl create configmap conf --from-env-file=env  
kubectl create configmap conf --from-literal=KEY=val
```

[stakater/Reloader](#) — auto-restart Deployment przy zmianie CM/Secret

→ `day2/05_configmap/`

Secrets — 3 typy (D2/10)

```
kubectl create secret generic      # Opaque
kubectl create secret docker-registry # imagePullSecrets
kubectl create secret tls          # cert + klucz dla Gateway/Ingress
```

⚠ Default = base64, NIE szyfrowane

Produkcja:

- etcd encryption at rest
- HashiCorp Vault (D4/04)
- SOPS (Git-encrypted)
- Sealed Secrets

→ day2/10_secrets/

AuthN/AuthZ flow

Request → AuthN → AuthZ → Admission → etcd

AuthN: kim jesteś?

- ServiceAccount Token (in-cluster)
- Client Cert (x509)
- OIDC (SSO)
- Webhook (custom)

AuthZ: co możesz?

- **RBAC** (najpopularniejsze)
- ABAC (legacy)
- Webhook

→ `day2/06_auth/`

RBAC w 30 sekund

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role                                # lub ClusterRole (cross-namespace)
metadata: { name: pod-reader, namespace: prod }
rules:
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list", "watch"]
---
kind: RoleBinding
subjects:
  - kind: User                            # lub ServiceAccount, Group
    name: alice
roleRef:
  kind: Role
  name: pod-reader
```

```
kubectl auth can-i list pods --as=alice
```


Strategie AuthN — kiedy?

Strategia	Kiedy	Lifetime
ServiceAccount BoundToken	in-cluster Pod	1h auto-refresh
Client cert (x509)	admin emergency	per cert (1y)
OIDC	human users (SSO)	1h refresh
Webhook	custom	depends

→ `day2/06_auth/01_token`, `02_cert`, `03_oidc`

Gateway API + cert-manager (D2/07)

Modern alternative dla Ingress (GA od K8s 1.29):

```
GatewayClass    ← infra admin
└─ Gateway      ← cluster operator
   └─ HTTPRoute  ← app dev
```

- **cert-manager** dla automatycznych Let's Encrypt certs

→ `day2/07_gateway_api/` (Envoy Gateway implementation)

Gateway API — przykład

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata: { name: my-app }
spec:
  parentRefs: [{ name: training-gateway }]
  hostnames: [app.example.com]
  rules:
    - matches: [{ path: { type: PathPrefix, value: / } }]
      backendRefs: [{ name: my-app, port: 80 }]
```

Filter native: URLRewrite, RequestHeaderModifier, RequestMirror, RequestRedirect — bez annotations hell

StatefulSet (D2/08)

	Deployment	StatefulSet
Pod names	random	app-0 , app-1 , app-2
Ordering	równoległe	sekwencyjne
DNS	tylko Service	pod-N.service.ns.svc
Storage	shared	per-Pod PVC (volumeClaimTemplates)
Scale down	random	najwyższy ordinal

Use cases: PostgreSQL, MongoDB, Cassandra, Kafka, ElasticSearch

 **Bazy w K8s — tak czy nie?** (link Google blog w README)

→ `day2/08_statefulsets/`

DaemonSet (D2/09)

Jeden Pod per node (per matching label).

Use cases:

- Log collector (Fluent Bit)
- Metrics agent (node-exporter, DCGM)
- CNI (Calico, Cilium)
- Security (Falco — D4/08)
- Storage agent (Longhorn)

```
nodeSelector:  
  nvidia.com/gpu.present: "true"    # tylko GPU nody (D5/07)
```

→ `day2/09_daemonsets/`

Podsumowanie D2

- ✓ Workloady: Jobs, CronJobs, StatefulSet, DaemonSet
- ✓ Storage stack: emptyDir → PV → StorageClass
- ✓ ConfigMaps + Secrets (3 typy)
- ✓ AuthN/AuthZ + RBAC
- ✓ Gateway API + cert-manager + Let's Encrypt

Jutro: Scheduling, autoscaling, NetworkPolicy, deployment strategies